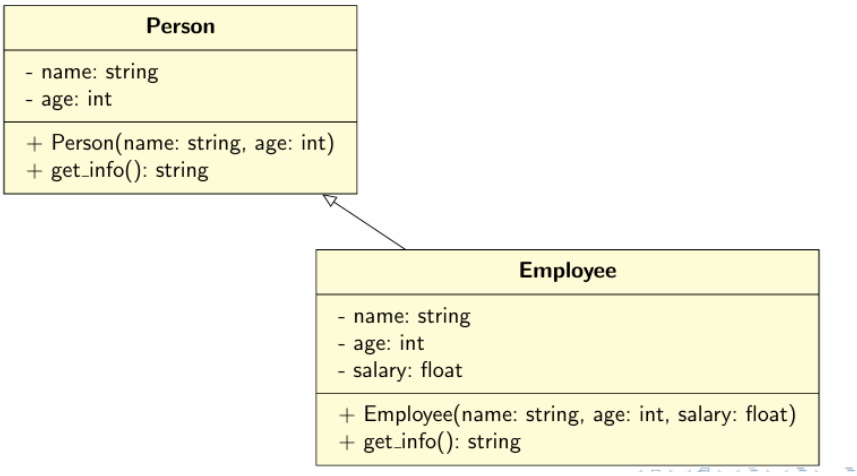
OOP

- Dědičnost
  - Odvozená třída:
    - Přebírá strukturu (atributy) a chování (metody) rodičovské třídy.
    - Může přidávat nové atributy a metody.
    - Může překrývat (upravit nebo předefinovat) metody rodičovské třídy.
  - V přetížené metodě lze volat metody rodiče klíčovým slovem **super()**
  - ```python
    class Person:
        def __init__(self, name, age):
            if age < 0:
                raise ValueError("Věk musí být větší než 0.")
            self.name = name
            self.age = age
        def get_info(self):
            return f"{self.name} ({self.age})."

    class Employee(Person):
        def __init__(self, name, age, salary):
            super().__init__(name, age) # rodičovský konstruktor
            self.salary = salary
        def get_info(self):
            return f"{self.name} ({self.age}) bere {self.salary} za rok."
    ```
    - 

- Abstraktní třídy a metody
  - Abstraktní třída
    - třída, která definuje rozhraní, ale neimplementuje všechny metody
  - Abstraktní metoda
    - metoda, která je deklarována, ale není implementována v abstraktní třídě.
  - abstraktní třídy a metody se vytvářejí pomocí modulu abc (Abstract Base Classes)
  - ```python
    from abc import ABC, abstractmethod

    class Animal(ABC):
      @abstractmethod
      def make_sound(self):
        pass

    class Dog(Animal):
      def make_sound(self):
        print("Woof")

    class Cat(Animal):
      def make_sound(self):
        print("Meow")
    ```

- Rozbalování parametrů

```python
def example_func(*args, **kwargs):
    print("Poziční argumenty:", args)
    print("Slovníkové argumenty:", kwargs)

example_func(1, 2, 3, a=4, b=5)
```

  -
    - ***args**
      - Používá se k zachycení libovolného počtu pozičních argumentů ve funkci. Argumenty jsou přístupné jako **n-tice (tuple)**.
    - ****kwargs**
      - Používá se pro zachycení libovolného počtu slovníkových (klíčových) argumentů. Argumenty jsou přístupné jako **slovník (dict)**.
- Návrhové vzory
  - obecné řešení často se vyskytujících problémů
  - Příklady návrhových vzorů
    - Tvorba objektů (creational)
      - Jedináček (singleton)
        - zajišťuje, že existuje pouze jedna instance dané třídy
      - Tovární metoda (factory method)
        - umožňuje vytvářet objekty bez specifikace konkrétní třídy
      - Abstraktní továrna (abstract factory)
        - umožňuje vytvářet rodiny souvisejících objektů bez specifikace konkrétních tříd
    - Organizace tříd a objektů (structural)
      - Adaptér (adapter)
        - umožňuje spolupráci dvou nekompatibilních rozhraní
      - Most (bridge)
        - odděluje abstrakci od implementace
      - Složenina (composite)
        - umožňuje pracovat se skupinou objektů jako s jedním objektem
    - Interakce mezi objekty (behavioral)
      - Pozorovatel (observer)
        - umožňuje objektům být informovány o změnách v jiných objektech
      - Strategie (strategy)
        - umožňuje definovat různé varianty algoritmu a měnit je za běhu
      - Šablona (template method)
        - umožňuje definovat kostru algoritmu a nechat podtřídy implementovat detaily